Module Guide for DAIIM

Team 6,
Robert Lam
Nathan Hum
Aidan Lao
Keira Laskoski

November 12, 2026

1 Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

2 Reference Material

This section records information for easy reference.

2.1 Abbreviations and Acronyms

symbol	description
AC	Anticipated Change
DAG	Directed Acyclic Graph
M	Module
MG	Module Guide
OS	Operating System
R	Requirement
SC	Scientific Computing
SRS	Software Requirements Specification
DAIIM	Explanation of program name
UC	Unlikely Change
[etc. —SS]	[... —SS]

Contents

1 Revision History **i**

2 Reference Material **ii**
2.1 Abbreviations and Acronyms ii

3 Introduction **1**

4 Anticipated and Unlikely Changes **2**
4.1 Anticipated Changes 2
4.2 Unlikely Changes 2

5 Module Hierarchy **3**

6 Connection Between Requirements and Design **4**

7 Module Decomposition **6**
7.1 Hardware Hiding Modules (M??) 6
7.2 Behaviour-Hiding Module 6
7.2.1 Input Format Module (M3) 6
7.2.2 Etc. 7
7.3 Software Decision Module 7
7.3.1 Etc. 7

8 Traceability Matrix **7**

9 Use Hierarchy Between Modules **9**

10 User Interfaces **9**

11 Design of Communication Protocols **10**

12 Database Design **10**

13 Timeline **10**

14 Generative AI Use Disclosure **10**
14.1 AI Tools Used 10
14.2 How and Where Used 10
14.3 Verification of AI-Generated Content 10

List of Tables

1 Module Hierarchy 4

2 Trace Between Requirements and Modules 8
3 Trace Between Likely Changes and Modules 8
4 Trace Between Unlikely Changes and Modules 9

List of Figures

1 Use hierarchy among modules 9

3 Introduction

Decomposing a system into modules is a commonly accepted approach to developing software. A module is a work assignment for a programmer or programming team (**?**). We advocate a decomposition based on the principle of information hiding (**?**). This principle supports design for change, because the "secrets" that each module hides represent likely future changes. Design for change is particularly valuable in software development, where modifications are frequent, especially during initial development as the solution space is explored. The system described in this document is the Decentralized AI Inference Marketplace (DAIIM), a web-based platform that allows users to submit inference requests and receive aggregated results from participating inference providers. The system uses blockchain-based programs to manage marketplace operations, including request lifecycles, provider participation, prediction aggregation, and token-based settlement. Supporting off-chain components provide services such as user interaction, transaction management, request input handling, and communication with inference providers. Our design follows the rules laid out by **?**, as follows:

- System details that are likely to change independently should be the secrets of separate modules.
- Each data structure is implemented in only one module.
- Any other program that requires information stored in a module's data structures must obtain it by calling access programs belonging to that module.

After completing the first stage of the design, the Software Requirements Specification (SRS), the Module Guide (MG) is developed (**?**). The MG specifies the modular structure of the system and is intended to allow both designers and maintainers to easily identify the parts of the software. The potential readers of this document are as follows:

- New project members: This document provides an overview of the system's modular structure, allowing new members to understand the overall design and quickly locate the modules relevant to their work.
- Maintainers: The hierarchical structure of the module guide helps maintainers understand the responsibilities and relationships between modules when modifications are required. Maintainers should update the relevant sections of this document when changes are made to the system.
- Designers: The module guide can be used to evaluate the consistency, feasibility, and flexibility of the system's decomposition. Designers can use it to verify that module responsibilities are appropriately separated and that anticipated changes can be accommodated without requiring unnecessary modifications to other modules.

The rest of the document is organized as follows. Section 4 lists the anticipated and unlikely changes to the system. Section 5 summarizes the module decomposition constructed

according to the information-hiding principle. Section 6 specifies the connections between the software requirements and the modules. Section 7 gives a detailed description of the modules. Section 8 includes two traceability matrices. One checks the completeness of the design against the requirements provided in the SRS. The other shows the relationship between anticipated changes and the modules. Section 9 describes the use relations between modules.

4 Anticipated and Unlikely Changes

This section identifies possible changes to DAIIM and classifies them according to their likelihood. Anticipated changes are listed in Section 4.1, while unlikely changes are listed in Section 4.2.

4.1 Anticipated Changes

Anticipated changes are design or implementation decisions that may reasonably need to be modified as the system develops. These decisions should be hidden within the modules responsible for them so that a change can be made with minimal impact on the rest of the system.

AC1: The types of AI inference tasks supported by DAIIM. The initial system focuses on regression tasks with numerical outputs, but support for additional inference types may be introduced in the future. The inference processing and aggregation components may need to be adapted to accommodate these additional types.

AC2: The algorithms used to aggregate provider predictions, update provider reputations, and calculate rewards. These mechanisms may be swapped out as the system is evaluated to improve prediction reliability, incentive fairness, and resistance to inaccurate or malicious providers.

AC3: The layout and presentation of the web-based user interface. The interface may change based on user feedback and usability testing, particularly regarding task creation, provider information, transaction status, and the presentation of aggregated results and prediction quality.

AC4: The implementation of off-chain supporting services and their communication mechanisms. The backend, database, and provider SDK may need to be updated or replaced as the system evolves to improve scalability, maintainability, or compatibility with external services.

4.2 Unlikely Changes

Although the system should accommodate anticipated changes, certain fundamental design decisions are not expected to change during the scope of the project. These decisions estab-

lish the core architecture and purpose of DAIIM, and changing them would likely require modifications across multiple parts of the system.

UC1: The fundamental purpose of DAIIM as a decentralized marketplace for requesting AI inferences from multiple independent providers and combining their predictions into an aggregated result. The system is not intended to become a centralized inference service controlled by a single provider.

UC2: The use of a public blockchain as an essential part of the system architecture. Marketplace operations such as request management, provider registration, prediction aggregation, reputation updates, and token settlement will continue to rely on on-chain execution and records rather than being controlled entirely by the off-chain backend.

UC3: The system's intended form as a web-based application. Task requesters and inference providers will continue to interact with DAIIM through a web interface or its provider-facing SDK, rather than requiring DAIIM to become a standalone desktop or mobile application.

5 Module Hierarchy

This section provides an overview of the module design. Modules are summarized in a hierarchy decomposed by secrets in Table 1. The modules listed below, which are leaves in the hierarchy tree, are the modules that will actually be implemented.

M1: Wallet Manager

M2: Chain Observer

M3: Request Input Manager

M4: User Interface

M5: User Information

M6: Provider Setup

M7: Request Listener

M8: Participation Client

M9: Provider Registry

M10: Request Lifecycle

M11: Settlement

M12: Token Manager

M13: Transaction Manager

Table 1: Module Hierarchy

Level 1	Level 2
Behaviour-Hiding Module	M1: Wallet Manager
	M2: Chain Observer
	M3: Request Input Manager
	M4: User Interface
	M5: User Information
	M6: Provider Setup
	M7: Request Listener
	M8: Participation Client
Software Decision Module	M9: Provider Registry
	M10: Request Lifecycle
	M11: Settlement
	M12: Token Manager
	M13: Transaction Manager

6 Connection Between Requirements and Design

The design of DAIIM is intended to satisfy the functional requirements developed in the SRS. The following key design decisions were made to address specific requirements:

- **Wallet Manager (M1):** To satisfy FR**??** and FR**??**, wallet authentication and transaction signing are handled through a dedicated wallet module. This module verifies ownership of wallet addresses and provides the interface for users to authorize transactions without exposing their private keys to DAIIM.
- **Chain Observer (M2):** To satisfy FR**??**, FR**??**, and FR**??**, a dedicated module monitors blockchain activity and retrieves transaction and account information. This separates reading and interpreting blockchain data from the modules responsible for initiating transactions and managing marketplace operations.
- **Request Input Manager (M3):** To satisfy FR**??**, FR**??**, FR**??**, and FR**??**, a dedicated module manages the secure storage, encryption, access, and deletion of inference request inputs. Input data is kept off-chain and made available only to the requester and providers that accepted the corresponding request.
- **User Interface (M4):** To satisfy FR**??**, FR**??**, FR**??**, and FR**??**, the web interface provides users with access to marketplace information, task creation, inference

requests, and aggregated results. It also presents transaction information and marketplace activity in a way that does not require users to interact directly with the blockchain.

- **User Information (M5):** To support FR?? and FR??, a dedicated module manages user profile information associated with verified wallet addresses. This allows user-specific information, such as provider activity and preferences, to be maintained separately from the underlying blockchain records.

- **Provider Setup (M6):** To support FR?? and FR??, a dedicated module handles the provider's initial connection to the blockchain and retrieves the provider's registration information for available tasks. This separates provider initialization and configuration from the ongoing handling of inference requests.

- **Request Listener (M7):** To satisfy FR?? and FR??, a dedicated module monitors inference requests that are relevant to a provider's registered tasks. It identifies new requests and communicates their details, payment information, and applicable deadlines to the provider.

- **Participation Client (M8):** To satisfy FR??, FR??, FR??, and FR??, a dedicated client module handles the provider's participation in inference requests. It retrieves authorized input data, generates the prediction commitment using a cryptographically secure salt, and submits commitments and reveals directly to the on-chain program.

- **Provider Registry (M9):** To satisfy FR??, FR??, and FR??, a dedicated module manages provider registration for individual inference tasks. It maintains provider eligibility, registration status, and task-specific staking requirements so that only eligible providers can participate in requests.

- **Request Lifecycle (M10):** To satisfy FR?? through FR??, a dedicated module manages the state and progression of inference requests. The request lifecycle is implemented as a state machine that controls the acceptance, commit, and reveal phases, enforces their deadlines, validates provider participation, and determines whether a request can proceed or must be cancelled.

- **Settlement (M11):** To satisfy FR??, FR??, FR??, and FR??, a dedicated module handles the finalization of inference requests. It calculates the aggregated result using the reputation-weighted median algorithm, updates provider reputations using the confidence-based reputation algorithm, and determines the distribution of rewards based on the resulting provider reputations.

- **Token Manager (M12):** To satisfy FR??, FR??, FR??, and FR??, a dedicated module manages token-related operations. These include transferring payments into escrow, managing provider stakes, releasing reserved funds, distributing tokens, and applying penalties such as slashing or burning when required.

- **Transaction Manager (M13):** To satisfy FR??, FR??, and FR??, a dedicated module coordinates the preparation and submission of user-authorized transactions. It validates transaction parameters, checks available balances, simulates transactions before submission, and tracks their status to prevent duplicate submissions or incorrect reporting of transaction outcomes.

7 Module Decomposition

Modules are decomposed according to the principle of "information hiding" proposed by **?**. The *Secrets* field in a module decomposition is a brief statement of the design decision hidden by the module. The *Services* field specifies *what* the module will do without documenting *how* to do it. For each module, a suggestion for the implementing software is given under the *Implemented By* title. If the entry is *OS*, this means that the module is provided by the operating system or by standard programming language libraries. *DAIIM* means the module will be implemented by the DAIIM software.

Only the leaf modules in the hierarchy have to be implemented. If a dash (–) is shown, this means that the module is not a leaf and will not have to be implemented.

7.1 Hardware Hiding Modules (M??)

Secrets: The data structure and algorithm used to implement the virtual hardware.

Services: Serves as a virtual hardware used by the rest of the system. This module provides the interface between the hardware and the software. So, the system can use it to display outputs or to accept inputs.

Implemented By: OS

7.2 Behaviour-Hiding Module

Secrets: The contents of the required behaviours.

Services: Includes programs that provide externally visible behaviour of the system as specified in the software requirements specification (SRS) documents. This module serves as a communication layer between the hardware-hiding module and the software decision module. The programs in this module will need to change if there are changes in the SRS.

Implemented By: –

7.2.1 Input Format Module (M3)

Secrets: The format and structure of the input data.

Services: Converts the input data into the data structure used by the input parameters module.

Implemented By: [Your Program Name Here]

Type of Module: [Record, Library, Abstract Object, or Abstract Data Type] [Information to include for leaf modules in the decomposition by secrets tree.]

7.2.2 Etc.

7.3 Software Decision Module

Secrets: The design decision based on mathematical theorems, physical facts, or programming considerations. The secrets of this module are *not* described in the SRS.

Services: Includes data structure and algorithms used in the system that do not provide direct interaction with the user.

Implemented By: –

7.3.1 Etc.

8 Traceability Matrix

This section shows the traceability between the system modules and the requirements, as well as between the modules and the likely and unlikely changes anticipated for DAIIM. These matrices demonstrate which modules are responsible for satisfying the requirements and which modules may need to be modified if the system evolves.

Table 2: Trace Between Requirements and Modules

Requirement(s)	**Modules**
FR??, FR??, FR??	M1, M5, M4, M13
FR??, FR??	M4, M9, M5
FR??–FR??	M4, M10, M13, M12
FR??, FR??, FR??, FR??	M9, M12, M13, M1, M6
FR??	M4, M5, M9, M2
FR??, FR??, FR??, FR??	M4, M10, M3, M12, M13, M7
FR??–FR??, FR??	M10, M9, M7, M12
FR??, FR??, FR??, FR??	M3, M8, M10
FR??, FR??–FR??, FR??–FR??	M10, M8, M3, M7
FR??, FR??, FR??	M10, M9, M12, M8
FR??, FR??, FR??, FR??	M11, M10, M4, M2
FR??, FR??, FR??	M11, M9, M12
FR??–FR??, FR??	M11, M10, M12
FR??–FR??	M9, M11, M12, M10
FR??, FR??, FR??, FR??	M13, M1, M2, M4, M12
SA??–SA??, SA??, SA??, SA??	M13, M1, M12, M4, M11
NFR??, NFR??	M3, M8
NFR??, NFR??	M8, M10
NFR??, NFR??	M11, M12
NFR??, NFR??	M10, M11, M9, M12
NFR??, NFR??, NFR??	M4, M13, M2
NFR??, NFR??, NFR??, NFR??	M4, M1, M5

Table 3: Trace Between Likely Changes and Modules

Likely Change (LC)	**Modules**
LC??	M6, M8, M10, M11
LC??	M9, M10, M11, M12, M7
LC??	M11, M9, M12
LC??	M7, M9, M10
LC??	M4, M5

Table 4: Trace Between Unlikely Changes and Modules

Unlikely Change (LC)	**Modules**
LC??	M4, M9, M10, M11, M8
LC??	M4, M1, M6

9 Use Hierarchy Between Modules

In this section, the uses hierarchy between modules is provided. A module A uses a module B if the correct execution of A depends upon the availability of a correct implementation of B. The arrows in the uses hierarchy are directed from the module that requires a service to the module that provides that service. For DAIIM, the uses hierarchy is organized around the User Interface, the provider-side SDK modules, the supporting off-chain modules, and the core on-chain modules. The User Interface (M4) depends on the Wallet Manager (M1), User Information (M5), Transaction Manager (M13), and Chain Observer (M2) to support wallet authentication, user information retrieval, transaction submission, and the display of blockchain state. On the provider side, the Provider Setup module (M6) uses the Wallet Manager (M1), User Information (M5), and Chain Observer (M2) to establish provider identity and retrieve registration information. The Request Listener (M7) uses the Chain Observer (M2) to monitor relevant requests and blockchain events. The Participation Client (M8) uses the Provider Setup module (M6), Request Input Manager (M3), and Transaction Manager (M13) to retrieve authorized request inputs and participate in inference tasks. The Transaction Manager (M13) uses the Wallet Manager (M1) and Chain Observer (M2) to prepare, authorize, and verify transactions. The core on-chain modules provide the functionality required to operate the marketplace. The Request Lifecycle module (M10) uses the Provider Registry (M9) and Token Manager (M12) to manage provider eligibility and request-related token operations. The Settlement module (M11) uses the Request Lifecycle module (M10), Provider Registry (M9), and Token Manager (M12) to calculate final results, update provider reputations, and distribute rewards or penalties. Figure 1 illustrates the uses relationships between the modules. The resulting hierarchy is a directed acyclic graph (DAG), with higher-level modules depending on lower-level modules that provide the services they require.

Figure 1: Use hierarchy among modules

10 User Interfaces

[Design of user interface for software and hardware. Attach an appendix if needed. Drawings, Sketches, Figma —SS]

11 Design of Communication Protocols

[If there is communication between hardware and software, it may be necessary to explain the design of the communication protocol. This section should be filled in with NA if it is not appropriate for a given project. —SS]

12 Database Design

[If a database is a significant component of the project, the database design should be summarized here. This section should be filled in with NA if it is not appropriate for a given project. —SS]

13 Timeline

[Schedule of tasks and who is responsible —SS]

[You can point to GitHub if this information is included there —SS]

14 Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

14.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

14.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

14.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.

- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]